SCOPE ALPHA

Behind The Scenes — Complete Technical Walkthrough

Everything you need to know to present Scope Alpha to a CS audience. This document walks through every layer of the application — from the landing page animations to the terminal data pipeline — explaining exactly which technology powers each feature and why each decision was made.

1. What Scope Alpha Is (Product Pitch)
2. The Landing Page — Hero, Animations, Floating Dashboard
3. The Terminal Shell — Sidebar, TopBar, Keyboard Shortcuts
4. Dashboard Page — Charts, Watchlist, Data Flow
5. Every Terminal Page (OI, Funding, Liquidations, Basis, Screener)
6. The Design System — NeuCard, DataCell, Badge, Colors
7. The Data Layer — Seeded PRNG, Mock Generators
8. State Management — Zustand + Hydration Strategy
9. Fonts, CSS, Tailwind Configuration
10. Full Tech Stack Summary
11. Is There a Live API? (Honest Answer)
12. Likely Questions & Answers (Cheat Sheet)
13. File Map — Every File and What It Does
14. How to Run It

March 2026 · Presentation Reference Document

1. What Scope Alpha Is

Scope Alpha is a **crypto derivatives analytics terminal**. Think of it as a Bloomberg Terminal but specifically for crypto futures and perpetual swap markets.

Who is it for?

- **Quantitative traders** who need funding rate signals and basis arbitrage data.
- **Risk managers** monitoring open interest shifts and liquidation cascades.
- **Derivatives desks** that need a unified view of all leveraged market signals.

What signals does it track?

- **Funding Rates** — the periodic fee perpetual swap holders pay. When longs pay shorts, the market is over-leveraged long.
- **Open Interest (OI)** — total value of outstanding futures contracts. Rising OI + rising price = new money entering.
- **Liquidations** — forced closures of leveraged positions. Cascading liquidations cause flash crashes.
- **Basis** — the premium or discount of futures vs spot price. Positive basis (contango) = bullish sentiment.
- **Health Score** — a composite metric combining all the above into a single risk regime indicator.

Key point for your presentation: This is a **frontend prototype** with no live data yet. All numbers are generated by a seeded random number generator. The UI is production-quality, but the data is fake (by design).

2. The Landing Page

When you open **localhost:3000**, you see the landing page. This is rendered by `app/page.tsx` which just imports `components/landing/LandingPage.tsx` — a single 376-line file that contains the entire landing experience.

2a. The Ambient Background (Mouse-Tracking Orbs)

Behind everything, there are two large blurred circles that float around:

- A **green orb** (600px wide, rgba(0,212,170,0.12)) that tracks your mouse position with a 2-second eased delay.
- A **blue orb** (400px, rgba(59,130,246,0.08)) that floats on its own using the `orbFloat` CSS keyframe animation.
- A **dot grid** overlay — using CSS `backgroundImage` with two linear gradients at 60px intervals creating a subtle graph paper effect.

Built with: Pure CSS + React state. The mouse position is tracked via `window.addEventListener('mousemove')`, stored in `useState`, and applied as inline `top`/`left` CSS with `transition: top 2s ease, left 2s ease` for the smooth lag effect.

2b. The Navigation Bar (Scroll-Aware)

The nav bar is **fixed** at the top. It starts fully transparent and becomes a frosted glass panel when you scroll past 50px:

- **Transparency to glass:** Controlled by `scrollY` state from a scroll event listener. When scrollY > 50, it applies `rgba(11,14,20,0.85)` background + `backdrop-filter: blur(20px) saturate(1.2)`.
- **Logo:** The Scope Alpha logo is rendered with Next.js `<Image>` component (automatic optimization, lazy loading).
- **'Launch Terminal' button:** Uses our custom `NeuButton` component with `primary` prop. Clicking it calls `router.push('/terminal')` (Next.js client-side navigation).

Built with: Next.js `useRouter` for navigation, React `useState`/`useEffect` for scroll tracking, CSS `backdrop-filter` for the glassmorphism effect.

2c. The Hero Section

The big headline area with staggered fade-in animations:

- **'DERIVATIVES INTELLIGENCE' pill:** A green-bordered pill with a pulsing green dot (CSS `pulse` animation).
- **'See Through The Market Noise' heading:** Uses the `Outfit` font at `clamp(40px, 7vw, 82px)` — this is CSS `clamp()` for responsive font sizing without media queries.

- **'Noise' shimmer effect:** The word 'Noise' has a `shimmer-text` CSS class that applies a moving gradient (`background: linear-gradient(90deg, #E2E8F0, #00D4AA, #3B82F6, #00D4AA, #E2E8F0)`) with `background-clip: text` and `animation: textShimmer 6s linear infinite`.
- **Staggered animation:** Each element uses `animation: fadeInUp 0.8s cubic-bezier(0.16,1,0.3,1) Xs both` where X increases (0.2s, 0.4s, 0.6s, 0.8s, 1s) for a waterfall effect.

Built with: CSS keyframe animations, CSS `clamp()` for responsive sizing, `-webkit-background-clip: text` for the gradient text effect.

2. The Landing Page (continued)

2d. The Floating Dashboard Mockup

This is the hero image — a fake terminal screenshot that floats and rotates in 3D:

- **3D perspective:** The parent div has `perspective: 1200px`. The inner div uses the `float` CSS animation: `translateY(0px) rotateX(8deg) rotateY(-4deg)` oscillating to `translateY(-16px) rotateX(6deg) rotateY(-2deg)` over 8 seconds.
- **Glow shadow:** `box-shadow: 0 40px 80px rgba(0,0,0,0.6), 0 0 120px rgba(0,212,170,0.06)` — a deep shadow plus a subtle green ambient glow.
- **macOS title bar dots:** Three small circles (#FF5F57 red, #FEBC2E yellow, #28C840 green) to mimic a macOS window.
- **Animated bar chart:** 20 bars with staggered `barGrow` animation (scaleY from 0 to 1), each delayed by `i * 0.04s` for a cascade effect. Bars alternate between green (bullish) and red (bearish) gradients.
- **SVG funding overlay:** An inline SVG with a curved `<path>` using quadratic Bezier curves and a gradient stroke.

Built with: The entire mockup is a React component (`DashboardMockup`) — no images, no screenshots. It's JSX rendering divs with inline styles, plus one inline SVG. The animation is pure CSS.

2e. Stats Counter Bar

The '20+ Assets / <500ms Latency / 14 Data Sources / 99% Uptime' section:

- **Counting animation:** A custom `Counter` component that uses `setInterval` at 16ms (~60fps) to increment from 0 to the target number over 2 seconds.
- **Trigger on scroll:** Uses a custom `useInView` hook built with the **Intersection Observer API**. The counter only starts when the element scrolls into view.
- **Gradient text:** The numbers use `background: linear-gradient(135deg, #00D4AA, #3B82F6)` with `-webkit-background-clip: text`.

2f. Features Grid

Six cards showing the core modules. Each card:

- Uses our **NeuCard** component with `glow` prop (adds green glow on hover).
- Has an icon in a gradient-bordered container with a subtle box-shadow glow matching the feature color.
- Fades in on scroll using the **Reveal** component (Intersection Observer + CSS opacity/transform transition with staggered delays).

2g. Tech Strip + CTA + Footer

- **Tech pills:** ['Next.js', 'FastAPI', 'TimescaleDB', 'Redis', 'Binance WS', 'CoinGlass', 'WebSocket', 'Python'] rendered as neumorphic-styled pills. Each fades in on scroll.
- **CTA card:** A large neumorphic card with a radial green glow behind the heading. 'Launch Scope Alpha' button navigates to /terminal.
- **Footer:** Logo + tagline, very minimal.

3. The Terminal Shell

When you click 'Launch Terminal', Next.js navigates to `/terminal`. This route has its own **layout** (`app/terminal/layout.tsx`) that wraps all 6 terminal pages with a persistent sidebar and top bar.

3a. Sidebar (components/layout/Sidebar.tsx)

- **Navigation:** 6 items — Dashboard (■), Funding (■), Open Interest (■), Liquidations, Basis (■), Screener (■). Each is a button that calls `router.push(path)`.
- **Active state:** Detected by comparing `usePathname()` to the item's path. Active items get a green left border + green tinted background.
- **Collapsible:** The sidebar width toggles between 220px and 56px. State is in Zustand (`sidebarCollapsed`) so it persists across refreshes.
- **Logo:** Next.js `<Image>` component loading `/logo.png` from the public folder.
- **Background:** CSS gradient: `linear-gradient(180deg, #0F1219 0%, #0B0E14 100%)`.

3b. TopBar (components/layout/TopBar.tsx)

- **Asset selector:** Shows the currently selected asset (e.g. 'BTC / USDT') from Zustand state. Has a dropdown arrow indicator (non-functional in prototype).
- **Timeframe pills:** Seven buttons (1m, 5m, 15m, 1H, 4H, 1D, 1W). Active pill has green tint + inset shadow (neumorphic pressed state). Inactive pills have raised shadow.
- **Connection status:** A pulsing green dot with 'Live' label. This is fake — `connectionStatus` is hardcoded to 'connected' in the Zustand store.
- **Settings gear:** An inline SVG icon (no functionality, just visual).
- **Glass effect:** `background: rgba(15,18,25,0.8)` + `backdrop-filter: blur(12px)`.

3c. Keyboard Shortcuts (components/terminal/KeyboardShortcuts.tsx)

- Press **?** to show a modal overlay listing all shortcuts.
- Press **1-6** to navigate directly to any page.
- Uses `window.addEventListener('keydown')` with `useCallback` to avoid stale closures.
- The overlay is a `fixed inset-0` div with `bg-black/60 backdrop-blur-sm` backdrop.
- Ignores keypresses when focus is in an input or textarea.

3d. ErrorBoundary (components/ui/ErrorBoundary.tsx)

Wraps the main content area. If any page throws a render error, it catches it and shows a recovery UI instead of crashing the whole terminal.

4. Dashboard Page

File: `app/terminal/page.tsx` (85 lines). This is the default view when you enter the terminal.

4a. Stats Bar (Top Row)

Four `DataCell` components showing: Health Score, Funding Rate, Open Interest, Basis (Annualized).

- Data comes from calling `generateFundingData(selectedAsset)`, `generateOIData(selectedAsset)`, `generateHealthScore(selectedAsset)`.
- All wrapped in `useMemo` keyed on `selectedAsset`, so they only recalculate when you switch assets.

4b. Candlestick Chart

The big chart is powered by **TradingView Lightweight Charts** — this is the only external charting library in the entire app.

- **Component:** `components/charts/PriceChart.tsx` (138 lines).
- **How it works:** On mount, it calls `createChart()` from the `lightweight-charts` npm package, attaches it to a div ref, and creates two series: a **CandlestickSeries** (price) and a **HistogramSeries** (volume).
- **Colors:** Green (#10B981) for bullish candles, Red (#EF4444) for bearish. Volume bars are 30% opacity versions of the same colors.
- **Styling:** Grid lines at rgba(255,255,255,0.03), crosshair at 0.15 opacity, JetBrains Mono font for axis labels.
- **Data:** Generated by `generateCandles(basePrice, count)` from `mockData.ts`. Returns OHLCV objects with realistic price action (drift + volatility model).
- **Resize handling:** A `window.addEventListener('resize')` that calls `chart.applyOptions()` with new dimensions.

4c. Watchlist (Right Panel)

Component: `components/terminal/Watchlist.tsx` (85 lines).

- Shows all 20 tracked instruments with rank, price, 24h change, and volume.
- **Sortable:** Click column headers to sort by rank, price, change, or volume.
- **Interactive:** Click any row to change `selectedAsset` in Zustand — the entire terminal updates reactively.
- Selected row gets a green left border + green background tint.

5. Every Terminal Page

Each page follows the same pattern: import data generators, call them with `selectedAsset` in `useMemo`, render with inline SVG charts + DataCells + tables.

5a. Open Interest (app/terminal/oi/page.tsx)

- **Big OI card:** Shows aggregate OI with 1h/4h/24h change indicators.
- **OI/Market Cap ratio:** Calculated inline as a leverage proxy.
- **Momentum:** Compares average of last 24 hourly values vs previous 24.
- **OI vs Price Correlation:** Deterministic estimate using a seed derived from OI and price changes.
- **7-day area chart:** Inline SVG (`MiniChart` component) — 168 data points, gradient fill, polyline stroke.
- **Exchange breakdown bar:** A horizontal stacked bar showing Binance (gold), OKX (white), Bybit (amber) proportions.
- **OI Leaderboard:** Table ranking all 20 instruments by OI size.

5b. Funding Rates (app/terminal/funding/page.tsx)

- 20-instrument funding rate table with z-score badges.
- Selected asset detail panel with SVG line chart of 90 funding rate data points.
- Exchange breakdown with horizontal bar comparison.

5c. Liquidations (app/terminal/liquidations/page.tsx)

- Scrolling liquidation event ticker.
- Long vs Short stacked bar with gradient fills.
- 24-bar hourly liquidation volume chart (SVG stacked bars).
- Top 10 largest liquidations table.

5d. Basis Monitor (app/terminal/basis/page.tsx)

- Large basis display with contango/backwardation signal.
- 30-day area chart (180 data points at 4h intervals).
- **Dual-line overlay:** A `DualLineChart` component that normalizes two datasets to [0,1] and overlays them — basis (solid) vs funding rate (dashed).
- Exchange basis comparison with progress bars.
- Basis leaderboard across all instruments.

5e. Screener (app/terminal/screener/page.tsx)

- Full-width sortable table with 11 columns + sparkline mini-charts.
- Click any header to sort. Active sort column highlighted in green.
- Health score cells have dynamically calculated background opacity based on score.
- Regime badges use `getRegimeColor()` to map 'Risk Off'/'Cautious'/'Neutral'/'Risk On'/'Overheated' to color schemes.
- Click any row to select that asset globally.

Key technical point: Every chart on every page is an **inline SVG** — there's no D3, no Recharts, no Chart.js. Each chart is a React component that calculates SVG coordinates from data arrays and renders <polyline>, <polygon>, <rect>, etc. This means **zero additional bundle size** for charts.

6. The Design System

6a. NeuCard (components/ui/NeuCard.tsx)

The primary container component used everywhere. 40 lines of code.

- **Background:** `linear-gradient(145deg, rgba(18,22,32,0.6), rgba(13,16,26,0.6))` — a subtle diagonal gradient.
- **Glassmorphism:** `backdrop-filter: blur(20px)`.
- **Neumorphic shadows:** Raised state = `6px 6px 20px rgba(0,0,0,0.4), -3px -3px 10px rgba(255,255,255,0.02)`. Two shadows, one dark (bottom-right) and one light (top-left), creating the 3D emboss effect.
- **Hover state:** Border changes from white/5% to green/20%, shadow deepens, optional green glow.
- **Props:** `glow` (adds green ambient glow), `noPadding` (for tables that need edge-to-edge content), `onClick` (adds pointer cursor + lift on hover).

6b. NeuButton (components/ui/NeuButton.tsx)

Neumorphic button with press animation. 51 lines.

- **Primary variant:** Green gradient background, dark text, green glow shadow.
- **Default variant:** Dark gradient background, light text, subtle shadow.
- **Press effect:** On mouseDown, shadow changes from raised to `inset` and scale drops to 0.97. This creates the physical 'pushed in' neumorphic feel.

6c. DataCell (components/ui/DataCell.tsx)

Standardized metric display. Used for every stat box across all pages.

- Shows: label (10px mono uppercase), value (18px bold mono, colored), optional change indicator, optional subtitle.
- Background: `bg-white/[0.02]` with `border border-white/[0.04]`.

6d. Badge (components/ui/Badge.tsx)

Colored pill labels. 6 variants: green, red, yellow, blue, purple, gray.

- Each variant maps to a Tailwind class string like `bg-emerald-500/15 text-emerald-400 border-emerald-500/20`.
- 10px monospace font, semibold, rounded corners.

6e. Color Palette

	#0B0E14	Background (base) - used on body and root
	#0F1219	Surface - sidebar, elevated panels
	#131620	Elevated - card interiors, table headers
	#00D4AA	Accent Green - primary actions, positive values, nav active
	#8B5CF6	Accent Purple - basis data, secondary accent
	#3B82F6	Accent Blue - OI data, informational
	#F59E0B	Accent Yellow - warnings, selected asset dot
	#EF4444	Negative - bearish, liquidations, errors
	#10B981	Positive - bullish candles, positive changes
	#EC4899	Accent Pink - health score, special highlights

7. The Data Layer

File: `lib/mockData.ts` (359 lines). This is the heart of the data pipeline.

7a. Seeded PRNG (mulberry32)

Instead of `Math.random()`, we use a **seeded pseudo-random number generator** called mulberry32. This is a well-known 32-bit hash-based PRNG.

```
function mulberry32(seed: number) { let s = seed | 0; return (): number => { s = (s + 0x6d2b79f5) | 0; let t = Math.imul(s ^ (s >>> 15), 1 | s); t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t; return ((t ^ (t >>> 14)) >>> 0) / 4294967296; }; }
```

- **Seed derivation:** Each asset symbol (e.g. 'BTC') is hashed to a number using `Math.imul(31, h) + charCode`. Then different data types append a suffix (e.g. 'BTCfunding', 'BTCoi') for unique seeds per data type.
- **Why?** Server-side rendering and client-side rendering must produce identical output. `Math.random()` gives different values each call, causing React hydration mismatches. Seeded PRNG gives the same sequence every time for the same seed.

7b. The 20 Instruments

Hardcoded in `INSTRUMENT_DEFS`: BTC, ETH, SOL, BNB, XRP, ADA, AVAX, DOGE, DOT, LINK, MATIC, UNI, ATOM, LTC, NEAR, APT, ARB, OP, FIL, INJ. Each has a realistic base price, market cap, and 24h volume.

7c. Mock Generators

Each generator takes a symbol string and returns structured data:

- **generateCandles(basePrice, count)** — Simulates OHLCV candles with drift + volatility model. Uses a fixed base timestamp (2025-03-17T00:00:00Z) to avoid server/client time differences.
- **generateFundingData(symbol)** — Current rate, 8h avg, z-score, annualized carry, 90-point history (30 days at 8h intervals), per-exchange breakdown.
- **generateOIData(symbol)** — Aggregate OI (1-5% of market cap), 1h/4h/24h changes, 168-point history (7 days hourly), per-exchange split.
- **generateLiquidationData(symbol)** — 50 liquidation events with random sides/amounts/exchanges, 24-bar hourly volume history.
- **generateBasisData(symbol)** — Current basis, annualized, 180-point history (30 days at 4h), per-exchange comparison.
- **generateHealthScore(symbol)** — Composite score from funding (30%), OI (25%), liquidation (25%), basis (20%) components. Maps to regime labels.

• **generateScreenerData()** — Calls all the above for every instrument and combines into a flat row for the screener table.

If someone asks 'can you swap in real data?': Yes. Each generator returns TypeScript interfaces (FundingData, OpenInterestData, etc.) that match what real exchange APIs return. You'd replace the generator function with an API call and the UI components wouldn't change at all.

8. State Management & Hydration

8a. Zustand Store (lib/store.ts)

The entire app has one global store with 4 pieces of state:

```
selectedAsset: string // e.g. 'BTC' selectedTimeframe: TimeFrame // e.g. '1D'
sidebarCollapsed: boolean connectionStatus: 'connected' | 'connecting' | 'disconnected'
```

- **Zustand** is a tiny state library (1.2KB). You call `create()` with a function that returns your state + actions. Components use `useAppStore()` hook to read/write.
- **Persist middleware:** Wraps the store so state is automatically saved to `localStorage` under the key `scope-alpha-state`. This means if you select ETH, close the browser, and come back, ETH is still selected.

8b. The Hydration Problem

This was the biggest technical challenge in the project:

- 1. Server renders the page with **default state** (selectedAsset = 'BTC').
- 2. HTML arrives at the browser. React 'hydrates' it — attaches event listeners to the existing HTML.
- 3. Zustand's persist middleware loads saved state from localStorage (selectedAsset = 'ETH').
- 4. React sees that the server HTML says 'BTC' but the client wants 'ETH'. **Hydration mismatch error.**

8c. The Solution: useHydrated() Hook

```
export function useHydrated() { const [hydrated, setHydrated] = useState(false);
useEffect(() => setHydrated(true), []); return hydrated; }
```

Every page that reads persisted state checks `if (!hydrated)` and shows a loading spinner. After `useEffect` fires (confirming hydration is complete), the real content renders with the correct persisted state.

`useEffect` never runs on the server, and it runs after the first client render. So the first render on both server and client shows the spinner (matching). Then the effect fires, hydrated becomes true, and the real content appears with the correct state.

If someone asks 'why not just disable SSR?': We could use Next.js `dynamic(() => import(...), {ssr: false})`, but that gives us no server HTML at all — worse for initial load and SEO. The useHydrated approach gives us server-rendered layout structure while deferring only the data-dependent content.

9. Fonts, CSS, Tailwind

9a. Three Fonts

• **Outfit** (display font) — Used for headings, titles, branding. Loaded via Next.js `next/font/google` with `--font-display` CSS variable.

• **Satoshi** (body font) — Used for body text. Loaded via Fontshare CDN link in `<head>`.

• **JetBrains Mono** (monospace) — Used for all data, numbers, tables, code-like content. Loaded via `next/font/google` with `--font-mono` CSS variable.

9b. globals.css (211 lines)

Contains:

• **CSS custom properties (variables):** All colors, shadows, radii defined as `--bg-primary`, `--accent-green`, etc.

• **13 keyframe animations:** heroGlow, float, barGrow, pulse, pulseRed, fadeInUp, textShimmer, orbFloat, gradientShift, scanline, slideInLeft, flashGreen/Red, shimmer, spin, ticker.

• **Utility classes:** `.shimmer-text`, `.skeleton`, `.glass-panel`, `.neu-raised`, `.neu-pressed`, `.glow-green`, `.flash-green`, `.flash-red`, `.nav-active`.

• **Custom scrollbar:** 6px wide, transparent track, white/8% thumb.

9c. tailwind.config.ts

Extends the default Tailwind theme with:

• 15 custom color tokens (all the greens, blues, purples, etc.)

• 3 font family stacks (display, body, mono)

• 4 custom border-radius values (8/12/16/20px)

• 4 custom box-shadow presets (raised, pressed, glow-green, card-hover)

10. Full Tech Stack Summary

What	Technology	Where It's Used
Framework	Next.js 14 (App Router)	File-based routing, server rendering, layout nesting
Language	TypeScript (strict)	Every file — 30+ type interfaces in types.ts
Styling	Tailwind CSS v3	All components, custom design tokens in config
CSS Animations	Pure CSS @keyframes	13 animations in globals.css — no Framer Motion
State	Zustand + persist	store.ts — global asset/timeframe/sidebar state
Candlestick Charts	TradingView Lightweight	PriceChart.tsx — only external chart lib
All Other Charts	Inline SVG (custom)	Sparklines, area, bars, heatmaps — zero bundle cost
Font Loading	next/font/google + CDN	Outfit, JetBrains Mono via Next.js; Satoshi via CDN
Image Optimization	next/image	Logo in sidebar and landing page
Navigation	next/navigation	useRouter, usePathname for client-side routing
Scroll Animations	Intersection Observer	Custom useInView hook for scroll-triggered reveals
Data Generation	Seeded PRNG (mulberry32)	mockData.ts — deterministic fake data
Hydration Guard	Custom useHydrated hook	All 6 terminal pages

Important: There is NO Framer Motion, no D3, no Chart.js, no Recharts, no Emotion, no styled-components. Animations are pure CSS keyframes. Charts are inline SVG. Styling is Tailwind utility classes + inline styles.

11. Is There a Live API?

No. There is no live API, no backend, no database, and no WebSocket connection.

Everything you see on screen is generated client-side by the seeded PRNG in `mockData.ts`. The 'Live' connection indicator is fake — it's hardcoded to 'connected' in the Zustand store.

The landing page lists technologies like 'FastAPI', 'TimescaleDB', 'Redis', and 'Binance WS' in the tech strip — these are **planned** technologies for the backend, not currently implemented.

What would the backend look like?

- **Data ingestion:** WebSocket connections to Binance, Bybit, OKX for real-time market data.
- **API server:** FastAPI (Python) serving REST endpoints and relaying WebSocket streams.
- **Storage:** PostgreSQL + TimescaleDB for time-series historical data.
- **Cache:** Redis for hot data (current rates, latest OI).
- **Compute:** Background workers for z-scores, regime classification, health scores.

How hard is the swap?

The mock data generators return TypeScript interfaces (`FundingData`, `OpenInterestData`, etc.) that exactly match what real exchange APIs return. To go live, you'd replace each `generate*()` call with a React Query hook fetching from the real API. The UI components wouldn't change at all — they just consume the typed data.

12. Likely Questions & Answers

Q: What framework is this built on?

Next.js 14 with the App Router. It's React under the hood, but Next.js gives us file-based routing, server-side rendering, automatic code splitting, and optimized font/image loading.

Q: What's the chart library?

TradingView Lightweight Charts for the candlestick chart on the dashboard. Every other chart (area charts, bar charts, sparklines, heatmaps) is hand-built with inline SVG — no library at all.

Q: How do the animations work? Is that Framer Motion?

No external animation library. All animations are pure CSS @keyframes defined in globals.css. The landing page uses 8 different keyframe animations (fadeInUp, float, barGrow, textShimmer, orbFloat, heroGlow, pulse, ticker). Scroll-triggered animations use the browser's Intersection Observer API via a custom useInView() hook.

Q: How does the floating dashboard work?

It's a React component called DashboardMockup that renders divs and one inline SVG. The float effect is a CSS animation: translateY(0) rotateX(8deg) oscillating to translateY(-16px) rotateX(6deg) over 8 seconds. The parent has perspective: 1200px for the 3D effect.

Q: Is the data real?

No. All data is generated by a seeded PRNG (mulberry32 algorithm). The seed is derived from each asset's symbol, so the same asset always produces the same data. This was a deliberate choice to avoid hydration mismatches between server and client rendering.

Q: What's hydration?

In Next.js, the server renders HTML first, then the browser 'hydrates' it by attaching React event listeners. If the server and client produce different HTML, React throws errors. We use a useHydrated() hook to defer data-dependent rendering until after hydration completes.

Q: What state management are you using?

Zustand with the persist middleware. It's a 1.2KB library that gives us a global store with automatic localStorage sync. When you select an asset, it persists across page refreshes.

Q: What about the styling? Is that a CSS framework?

Tailwind CSS v3 with a custom configuration. We defined 15 custom colors, 3 font families, custom shadows, and border radii. The neumorphic effects (raised/pressed shadows) are a combination of Tailwind classes and inline styles.

Q: Can this handle real-time data?

The frontend architecture supports it. The mock data layer returns TypeScript interfaces that match real exchange API shapes. You'd swap the generator functions for WebSocket-fed React Query hooks. The UI components wouldn't need to change.

Q: How many lines of code is this?

Roughly 3,500 lines of TypeScript/TSX across 34 source files. The largest file is mockData.ts at 359 lines.

Q: Why not use Redux?

Zustand is simpler for our use case. We have 4 pieces of global state. Redux would add boilerplate (actions, reducers, selectors, provider) for no benefit. Zustand's persist middleware also handles localStorage automatically.

Q: Why inline SVG instead of a chart library?

Zero bundle size. A chart library like Recharts adds ~50KB. Our SVG charts are part of the component JSX — they add nothing to the bundle. Plus we get pixel-perfect control over every visual detail to match the neumorphic design language.

13. File Map — Every File and What It Does

App Routes

app/layout.tsx Root layout: loads fonts, global CSS, sets metadata
app/page.tsx Entry point: renders LandingPage component
app/globals.css CSS variables, 13 keyframe animations, utility classes
app/terminal/layout.tsx Terminal shell: Sidebar + TopBar + ErrorBoundary
app/terminal/page.tsx Dashboard: candlestick chart + watchlist + stats
app/terminal/oi/page.tsx Open Interest: area chart + exchange bar + leaderboard
app/terminal/funding/ Funding rates: table + history chart + exchange breakdown
app/terminal/liquidations Liquidations: ticker + stacked bars + event table
app/terminal/basis/ Basis monitor: area chart + dual-line + leaderboard
app/terminal/screener/ Screener: full sortable table + sparklines

Components

components/landing/LandingPage.tsx 376 lines — entire landing experience
components/charts/PriceChart.tsx 138 lines — TradingView wrapper
components/layout/Sidebar.tsx 81 lines — collapsible nav sidebar
components/layout/TopBar.tsx 87 lines — asset selector + timeframes
components/terminal/Watchlist.tsx 85 lines — sortable asset table
components/terminal/KeyboardShortcuts 87 lines — ? overlay + 1-6 nav
components/ui/NeuCard.tsx 40 lines — neumorphic container
components/ui/NeuButton.tsx 51 lines — neumorphic button
components/ui/DataCell.tsx 29 lines — metric display cell
components/ui/Badge.tsx 29 lines — colored pill label
components/ui/ErrorBoundary.tsx — render error catcher
components/ui/Skeleton.tsx — loading placeholder

Library

lib/store.ts 30 lines — Zustand store + persist config
lib/types.ts 108 lines — all TypeScript interfaces
lib/utils.ts 53 lines — formatPrice, formatPercent, cn(), colors
lib/mockData.ts 359 lines — seeded PRNG + all data generators
lib/useHydrated.ts 8 lines — hydration guard hook

Config

tailwind.config.ts 54 lines — custom colors, fonts, shadows
next.config.js — Next.js configuration
tsconfig.json — TypeScript strict mode

```
package.json — dependencies + scripts
```

14. How to Run It

Requires **Node.js 20+**. No environment variables, no database, no external services.

```
cd scope-alpha/frontend npm install npm run dev # Open http://localhost:3000
```

Or on this machine, just double-click `Start_Scope_Alpha.bat` in the Alpha folder.

The landing page loads first. Click 'Launch Terminal' to enter the terminal. Use the sidebar or press 1-6 to switch between views. Click any asset in the watchlist or screener to change the selected instrument — all views update reactively.

End of document. You've got this.